

Readerboard and Busylight User's Guide

Includes details on assembly, installation, and programming these messaging units.



The information in this document, and the hardware and software it describes, are hobbyist works created as an educational exercise and as a matter of personal interest for recreational purposes.

It is not to be considered an industrial-grade or retail-worthy product. It is assumed that the user has the necessary understanding and skill to use it appropriately. The author makes NO representation as to suitability or fitness for any purpose whatsoever, and disclaims any and all liability or warranty to the full extent permitted by applicable law. It is explicitly not designed for use where the safety of persons, animals, property, or anything of real value depends on the correct operation of the software.

As an educational exercise, it is assumed that the user has the necessary skill to appropriately set up, build, and use this project. Do not proceed to use, whether exactly as provided in the repository or with modifications of your own, unless you know exactly what you're doing and have verified that your actions and the product materials are suitable for your purpose and function as expected.

For Readerboard hardware (PCB) version 3.4.2, Busylight hardware version 2.2.1 (light tree version 1.1), and firmware version 2.3.8.

(Experimental support for Adafruit 16×32, 32×32, 16×32×2, and 64×64 RGB matrix displays is a work in progress with current PCB revision 0.0.0.)

Copyright © 2023, 2024 by Steven L. Willoughby (aka MadScienceZone), Aloha, Oregon, USA. All Rights Reserved. This document is released under the terms and conditions of the Creative Commons “Attribution-NoDerivs 3.0 Unported” license. In summary, you are free to use, reproduce, and redistribute this document provided you give full attribution to its author and do not alter it or create derivative works from it. See

<http://creativecommons.org/licenses/bynd/3.0/>

for the full set of licensing terms.



CONTENTS

Contents	iii
List of Figures	vi
List of Tables	vi
1 Hardware	1
Version 1	1
Version 2	1
Version 2	2
Version 3	2
Firmware Implications	3
Construction	3
Busylight Devices	6
Controller for Adafruit Matrices	9
2 Protocol Description	13
Web Service API	13
Direct Host-to-Device Communication	14
USB vs. RS-485	15
Command Summary	17
%T@—Run Full Lamp Test Pattern	18
*—Strobe Lights in Sequence	18
<—Scroll Text Across Display	20
=—Set Operational Parameters	20
=#—Set Serial Number	21
=*—Display POST Banners	22
=&—Persist Settings in EEPROM	22
?—Query Discrete LED Status	23
@—Set Column Cursor Position	25
A—Select Font	25
B—Beep/Play Sound Sequence	25
C—Clear Matrix Display	28
D—Set Dimmer Levels	29
F—Flash Lights in Sequence	29

H—Draw Bar Graph Data Point	31
I—Draw Bitmap Image	31
K—Set Current Color	32
L—Light Multiple LEDs	34
Q—Query Readerboard Status	34
M—Morse Code	36
S—Light Single LED	36
T—Display Text	37
V—Manipulate Segmented LED Display	38
X—Turn off Discrete LEDs	39
3 Server-Only Commands	41
current—Report Current Display State	41
post—Post a Message to the Displays	42
postlist—Report of Message Queue	44
unpost—Remove a Posted Message	44
update—Update Dynamic Content	45
4 Pinouts	47
8p8c RS-485 Connectors	47
DIN-8 Busylight Tree Connectors	48
Connectors for Version 3.4 Boards	49
Adafruit Matrix Versions	51
Connectors for Legacy Boards	52
5 Installing and Using the Readerboard and Busylight Units	55
Power	55
Standalone Units	56
Networked Units	56
Compiling the Supporting Software	56
Setting up rbserver	56
Setting up Per-User Utilities	56
Diagnostic Displays	56
6 Font Glyphs	57
Font #0 standard.font	57
Font #1 standard_variable.font	66
Font #2 symbol.font	75
Font #3 double.font	83
Font #4 led16.font	87
Font #5 led16a.font	90
Default Glyph	92
Font Definition Files	92
7 Preparing and Updating Firmware Images	95

8 Schematics and Diagrams

LIST OF FIGURES

4.1	Female 8p8c Modular Jack with T-568B Color Codes	48
4.2	DIN-8 Jack (looking into socket from front)	49

LIST OF TABLES

1.1	Bill of Materials	4
1.2	Bill of Materials (Busylight)	8
1.3	Bill of Materials for AdaSheild Board	10
1.4	Bill of Materials for AdaSheild Board	11
2.1	Examples of RS-485 Escape Bytes.	18
2.2	Summary of All Commands	19
2.3	Control Codes in String Values	21
2.4	Baud Rate Codes	22
2.5	ASCII Encoded Integer Values (0–63)	24
2.6	Font Codes	25
2.7	Font Table for Fonts #0 and #1	26
2.8	Font Table for Font #2	27
2.9	Font Table for Font #3	28
2.10	Discrete LED Codes and Colors	30
2.11	Transition Effect Codes	33
2.12	Color codes for <i>rgb</i> parameters	33
2.13	Alignment Codes	37
3.1	Transition codes for use in <code>post</code> command	42
3.2	Alignment codes for use in <code>post</code> command	42
3.3	Color codes for use with the <code>post</code> command	43

<i>List of Tables</i>	vii
3.4 Special Codes in Post Messages	44
7.1 Firmware Configuration Values	96

HARDWARE

People who are really serious
about software should make
their own hardware.

—Alan Kay

AS A HOBBY PROJECT, the design of the hardware and software for the readerboard project “grew in the telling” as new ideas sprang to mind. As of this writing, there are three different models of hardware prototypes which were designed and created.

Version 1

The initial prototype (PCB 1.0.1) was a fairly large board, approximately $20\frac{5}{8} \times 6\frac{3}{8}$ ” with an LED pitch of $\frac{1}{8}$ ”.

It requires an Arduino Mega 2560 or Arduino Due to drive it. A custom-made shield board attaches to the Arduino, which provides connectors for power and ribbon cables to drive the display (one to control the matrix, the other for the 8 status LEDs which appear in a horizontal row along the bottom right of the matrix).

Version 2

This version was based on the 1.0 design, but shrunk to a board size of about $18\frac{3}{16} \times 5\frac{3}{16}$ ” with an LED pitch of $\frac{1}{4}$ ”. It also replaced the obsolete TPIC6B595 with the newer STPIC6D595 chip. Further, it relocated the ribbon cable which

connects the Arduino controller to the LED matrix, moved the eight status LEDs to a vertical column to the right of the matrix, and switched to resistor arrays for the matrix current limiting resistors, instead of the individual discrete resistors used on the 1.0.1 board.

Version 2

Version 2.1 was a significant change to the 2.0 board in terms of how it interfaces with the Arduino, although it had the same size and physical layout as the 2.0 board.

This one eliminates the need for the ribbon cables and shield board. Instead, the Arduino controller mounts directly to the back of the display board. This means that the lower portion of the left edge of the enclosure has all of the external connections in one place, including power, RS-485 in and out, and the USB connector(s) on the Arduino itself.

Version 3

Revision 3.x of the hardware expands on the version 2.1 design by replacing the single-color LEDs with RGB LEDs arranged into 24 logical rows—each of the eight physical rows is logically a row of red, green, and blue LED elements.

Note that the discrete status LEDs are mislabeled on the revision 3.2.2 PCB. L_0 should be the LED on the bottom (canonically green) and L_7 is on top (white).

Version 3.4.1 (current model)

Version 3.4 added support for playing sounds on a speaker and added a diode (D520) to prevent the Arduino from trying to power the LEDs by itself if it is powered from the USB port but the external 9 V supply is not active.

Version 3.3.1

Version 3.3 of the hardware corrects some minor issues with the 3.2.2 revision by adding a cutout to avoid colliding with the connectors on the Arduino board and adjusting the mounting holes slightly. It also corrects the mislabeled L_0 – L_7 LEDs

Additionally, it adds R51 for Arduino Due reset circuits, J3 to manually reset the Arduino, and J4, R52, R53, and U16 to support the addition of a external EEPROM chip for microcontrollers (such as the Due) which don't have one onboard.

This was further improved with revision 3.3.1 (the current version), which replaced terminal block J1 with a higher-density terminal which

allows easier interconnection of two CAT6 cables without the need for off-board splices. The power supply now connects to a new terminal block J5. The power to the RS-485 transceiver chip is also now switchable to either 3.3 V or 5 V so it can be appropriately matched to the operating voltage of the Arduino board without damaging the Arduino's input ports.

Firmware Implications

The boards before PCB revision 3.3.0 are not supported by the firmware as currently stored in the project repository. They were prototype ideas which were never built, so this shouldn't be a problem.

There are compile-time switches documented starting on p. 95 which adjust the firmware image for different microcontrollers, default settings—or the only settings if your hardware does not include an EEPROM memory—and possibly, in the future, different board revisions.

Construction

Once you have created a printed circuit board (PCB) from the fabrication files included with the readerboard Git repository and obtained the parts as listed in Figure 1.1, carefully solder the components in place.

Warning! Danger! Only perform the assembly operation if you are qualified to do so. Soldering electronic components may be hazardous due to the high temperatures, sharp objects, electrical voltages, potentially toxic materials, and other dangers. This requires skill and expertise as well as personal protection equipment to perform safely. Please also observe all precautions to avoid damaging the electronic components themselves from harm due to heat or static electricity discharge.

Suggested Order

For best results, solder components in a way that allows best access to the solder points without some parts getting in the way of connecting subsequent ones. For example:

1. Solder all the surface mount chips first: U0–U15 (and optionally, U16).
 - U0, U12, U13: CD74HCT238PWR
 - U1, U2, U14, U15: ULN2803CDWR (ULN2803A SOP-18 packages may be used if aligned to the top 18 pads, leaving the bottom two empty, if they have otherwise identical pinouts and characteristics)
 - U3–U10: STPIC6D595MTR

Part No.	Qty	Mouser	Description
	1		Printed circuit board rev 3.2.2
	1		Arduino Due or Mega 2560
C0–9,11,12	12	810-FA18X7R1H10400	0.1 μ F capacitor
C10	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
C13	1		100 μ F electrolytic capacitor
D0–511	512	[1]	LED, 5mm, RGB, common cathode
D512	1		LED, 5mm, green
D513–14	2		LED, 5mm, amber (yellow)
D515–16	2		LED, 5mm, red
D517–18	2		LED, 5mm, blue
D519	1		LED, 5mm, white
D520	1		power diode, 4 A
J0	1		Arduino stacking pin set
J1a,b	2	179-TBLH10-381-08BK	8-position terminal block
J2,3	2	649-69190-102HLF	2-position jumper header
J4	1		3-position jumper header
J5,6	2		2-position Euro-style terminal block
P0–2	3		2-position jumper shunt
Q0–23	24	942-IRF9530NPBF	IRF9530 p-channel MOSFET
Q24	1		2N2222 or BC547 NPN transistor
R0–7,35–51	24	604-MFR-24FRF521K	1K resistor
R8–23	16		4 $\times$ 360 Ω resistor array, isolated
R24,29–31	4	603-MFR-25FTE52-300R	300 Ω resistor
R25–28	4		360 Ω resistor
R32–34,52–54	6	603-MFR-25FRF5210K	10K resistor
R _{TERM}	1		120 Ω resistor
U0,12,13	3	595-CD74HCT238PWR	CD74HCT238PWR non-inverting 3-to-8 decoder
U1,2,14,15	4		ULN2803CDWR octal NPN Darlington array
U3–10	8		STPIC6D595MTR power 8-bit shift register
U11	1	595-THVD1439VDR	THVD1439 half-duplex RS-485 driver/receiver
U16	1	579-24LC256T-E/SN	AT24C256 EEPROM
	24		TO-220 heat sinks for Q0–23 (optional)
	1		9V 2A DC power supply

[1] I found these at Amazon.com: [EDGELEC 5mm RGB Tri-Color 4Pin LED Diodes Common Cathode](#).

Table 1.1: Bill of Materials

- U11: THVD1439DR (Note that this chip is oriented 180° from the other chips on the board, with pin 1 to the lower right.)
 - If needed for your microcontroller (Arduino Due, usually), solder U16: AT24C256 EEPROM chip.
2. Solder the discrete resistors: R0–R7, R24–R51, and R54 (and optionally, R52 and R53). See below first to ensure the correct values are used for R24–R31 based on your choice of corresponding LEDs.
 - R0–R7, R35–50: 1K
 - R24, R29–R31: 300Ω
 - R25–R28: 360Ω
 - R32–R34: 10K
 - If U16 is installed, solder in R52–R53: 10K
 - If using sound output, solder in R54: 10K
 3. Solder the 0.1 μF capacitors: C0–C9, C11–C12.
 4. Solder the resistor arrays: R8–R23.
 5. Solder the RGB LEDs D0–D511. Ensure that the common cathode goes to pin 2 so that pins 1, 3, and 4 are respectively the red, green, and blue anodes.
 6. Solder the single-color LEDs D512–D519 in the colors of your choice, being mindful of the polarity. The anode has a square pin on the PCB. Our recommendation:
 - D512 (bottom): green (with R24=300Ω)¹
 - D513: amber² (with R25=360Ω)
 - D514: amber (with R26=360Ω)
 - D515: red (with R27=360Ω)
 - D516: red (with R28=360Ω)
 - D517: blue (with R29=300Ω)
 - D518: blue (with R30=300Ω)
 - D519 (top): white (with R31=300Ω)
 7. **If you are using an Arduino Due:** solder R51 (1K).

¹Resistor values based on typical LED voltage and current ratings. Check to see what is appropriate for your components.

²In deference to the effort my high school drama teacher went through to get us to refer to any yellow stage lights and gels as “amber” and the preference to using “amber” for signal lamp colors in other contexts and regions, both terms will be used interchangeably in this project.

8. **If you are using an Arduino Mega 2560:** Solder the 10 μ F capacitor C10, being mindful of the correct polarity. **Do not solder both R51 and C10. Only one or the other will be appropriate for a given microcontroller.**
9. Solder the Arduino bus stacking pins (six connectors—1 $\times$ 10-pin, 5 $\times$ 8-pin—all labeled collectively as “J0”). Ensure the pins extend down and out the back of the board.
10. Solder the 2-pin headers J2 and J3, and the 3-pin header J4.
11. Solder the terminal blocks J1 (which is actually a pair of 8-pin terminal blocks J1a and J1b), and J5.

Option: Using Single-Color LEDs

If you don’t need RGB LEDs for your application, the same PCB may be used with single-color 5mm LEDs instead. Simply solder them onto the board so that their cathode is on pin 2 and the anode is on pin 4 of the LED footprint. This means the LEDs will electrically all be connected to the blue circuit.

Since they won’t be used, omit parts U0, U2, U12, U14, C0, C11, Q0–Q15, R0–R7, and R35–R42.

Set the firmware value `HW_MODEL` to `MODEL_3xx_MONOCHROME` and recompile the firmware. This changes the behavior of the controller so that it understands it can only drive one color which is attached to the blue color plane. Now, all colors (other than “off”) will turn the LEDs on.

Busylight Devices

The readerboard units are designed to be compatible with another project by the author. The “Busylight” is a small stand-alone indicator which can be used to indicate simple statuses like room occupancy or whether someone is busy, free, on a phone call, etc.

The busylight can drive up to seven LEDs. If directly connected to a host computer via USB, it can be powered entirely from the USB port. Starting with version 2, it can also receive commands via RS-485 network. In that case it can be powered by a 9 V supply over the same CAT5, CAT5e, or CAT6 cable carrying the serial communications.

Firmware

Starting with revision 2 of the Busylight, it is driven by a firmware image compiled from the same source code as the Readerboard units. Setting the compiler preprocessor symbol `HW_MODEL` to `MODEL_BUSYLIGHT_2` will cause the Arduino compiler to prepare the image for a Busylight, excluding the

code that drives the readerboard-specific functions. In such firmware images, `HW_MC` should be set to `HW_MC_PRO` since the Busylight is based on the SparkFun Arduino-compatible Pro Micro microcontroller.

Also set the `SERIAL_VERSION_STAMP` symbol to reflect the device serial number, hardware and firmware revision numbers. See Chapter 7 starting on p. 95 for more information about preparing firmware images.

Construction

Once you have created a printed circuit board (PCB) from the fabrication files included with the Git repository and obtained the parts as listed in Figure 1.2, carefully solder the components in place.

Warning! Danger! Only perform the assembly operation if you are qualified to do so. Soldering electronic components may be hazardous due to the high temperatures, sharp objects, electrical voltages, potentially toxic materials, and other dangers. This requires skill and expertise as well as personal protection equipment to perform safely. Please also observe all precautions to avoid damaging the electronic components themselves from harm due to heat or static electricity discharge.

Suggested Order

For best results, solder components in a way that allows best access to the solder points without some parts getting in the way of connecting subsequent ones. For example:

1. Solder the surface mount components:
 - U0 ULN2003A 7-circuit NPN Darlington transistor array. Note that the board was designed for narrow SOIC-16 packages but there is an additional column of pads for a wider version of this chip.
 - U1 THVD1439 RS-485 driver/receiver.
2. Solder through-hole components:
 - R0–R2 10K resistors
 - C0 0.1 μ F capacitor
3. Solder the sockets:
 - J0 8-pin DIN socket
 - J1 16-circuit Euro-style terminal block (alternatively you may choose to directly solder the CAT6 wires into the holes on the PCB.
 - Arduino stacking pins.
4. Place the microcontroller board onto the stacking pins.

Part	Qty	Description
	1	Printed circuit board rev 2.1
	7	LED tier printed circuit board rev 1.1
	1	SparkFun Pro Micro Arduino-compatible μ C, 5 V version
C0	1	0.1 μ F capacitor
C1	1	100 μ F capacitor
D0	1	power diode, 1 A
J0	1	8-pin DIN connector, PCB mount
J1	1	16-pin Euro-style terminal block
J2	1	Arduino stacking pin set
J3	1	2-pin Euro-style terminal block
P0	1	8-pin DIN plug
Q0	1	BC547 or 2N2222 NPN transistor
R0–3	4	10K resistor
U0	1	595-ULN2003AIDR ULN2003A NPN Darlington array
U1	1	THVD1439 half-duplex RS-485 driver/receiver
	14	5-pin bussed resistor network (2/LED tier)*
	56	LED, 5 mm or 3 mm, through-hole (8/LED tier)†
	1	$\frac{1}{4}$ " screw rod
	1	Enclosure‡
	8	Nylon spacers between (and above and below) the tiers
	2	$\frac{1}{4}$ " hex nuts

*Choose each tier's pair of resistor arrays appropriately for the voltage drop for the LEDs to be used on that tier. Assuming a 5 V supply and typical LED characteristics for a desired 20 mA current draw, that would be 150 Ω or larger for red and amber LEDs and 100 Ω or larger for blue, green, and white LEDs. Check your LED datasheet to confirm. Examples: Mouser 652-4605X-1LF-100 (100 Ω) or 652-4605X-1LF-150 (150 Ω).

†Choose LED colors as desired for each tier (standard is green, 2 $\times$ amber, 2 $\times$ red, blue, and white).

‡One idea for an inexpensive enclosure is a clear plastic tube storage container with an inside diameter large enough to hold the LED tier boards.

Table 1.2: Bill of Materials (Busylight)

5. Build each LED tier of 8 LEDs:

- Solder eight 3 mm LEDs around the edge of the board with their leads straddling the edge, so the anode is on the side of the PCB labelled “anode side” and the cathode is on the “cathode side.” Solder the leads to the exposed pads and trim the leads so they don’t extend past the pads.
- Solder the two resistor arrays, making sure to select the correct values for the specific LEDs used on this tier board.

6. Assemble the tier stack by threading the tiers onto the $\frac{1}{4}$ " threaded rod with nylon spacers between them, secured in place with hex nuts on either end of the stack. The following assumes the use of a length of CAT6 cable between the PCB and the light tree stack.

- Solder one end of the cable to P0, the 8-bin DIN plug. Suggested color assignments:
 - Pin 1 (sixth tier/blue): blue
 - Pin 2 (third tier/amber): brown/white
 - Pin 3 (fifth tier/red): orange/white
 - Pin 4 (fourth tier/red): orange
 - Pin 5 (seventh tier/white): blue/white
 - Pin 6 (second tier/amber): brown
 - Pin 7 (+5 V supply): green
 - Pin 8 (bottom tier/green): green/white
- Strip the insulation from the green wire to the height of the assembled tier tree. As each tier is added, thread this bare wire through the “V+” hole of each tier, soldering it to each tier along the way.
- Attach the wire for each tier to its “GND” hole.
- Thread the remaining wires through the larger holes to pass them up to the next tiers.

Controller for Adafruit Matrices

We obtained a number of pre-made RGB LED matrix displays from Adafruit which have a similar (but not quite the same) control logic to our own design, which allows us to also control those boards using only a slight variation in our hardware and firmware design.

The hardware to drive them is a small shield board that plugs onto an Arduino Due or Mega 2560. It incorporates the same control circuitry as our original readerboard circuit described above, without the actual LEDs themselves or the discrete chips that drive the rows and columns of the display, since those are already on the pre-made displays.

This shield provides two ribbon cable connectors: J7, a 16-pin cable which sends the control signals to the matrix display, and J8, a 10-pin cable that can drive an 8-LED discrete status display such as is on the busylight and our original readerboard.

The following describes the assembly of one of these shield boards.

AdaShield/QS Variation

The author produced a variation on the AdaShield boards for another project, but they may still be useful for general use as an alternative to the above-described board in that it lacks a speaker output, but adds an I²C alphanumeric LED display output and expands the discrete LED output from 8 LEDs to 14.

Part	Qty	Mouser	Description
	1		AdaShield Printed circuit board rev 0.0.0
	1		Arduino Due or Mega 2560
C0	1		100 μ F electrolytic capacitor
C1	1	810-FA18X7R1H10400	0.1 μ F capacitor
C2	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
D0	1		power diode, 1 A
J0	1		Arduino stacking pin set
J1a,b	2		8-position terminal block
J2,3	2		2-position jumper header
J4	1		3-position jumper header
J5,6	2		2-position Euro-style terminal block
J7	1		16-position IDC header
J8	1	517-89110-0101	10-position IDC header
P0–2	3		2-position jumper shunt
Q0	1		2N2222 or BC547 NPN transistor
R0–5	6	603-MFR-25FRF5210K	10K resistor
R6	1	604-MFR-24FRF521K	1K resistor
R _{TERM}	1		120 Ω resistor
U0	1		THVD1439 half-duplex RS-485 driver/receiver
U1	1		AT24C256 EEPROM
U2	1		ULN2803CDWR octal NPN Darlington array
	1		9V 2A DC power supply

Table 1.3: Bill of Materials for AdaSheild Board

Part	Qty	Mouser	Description
	1		AdaShield/QS Printed circuit board rev 0.0.0
	1		Arduino Due or Mega 2560
C0	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
C1	1	810-FA18X7R1H10400	0.1 μ F capacitor
D0	1		power diode, 2 A
J0	1		Arduino stacking pin set
J1a,b	2		8-position terminal block
J2,3	2		2-position jumper header
J4	1		3-position jumper header
J5	1		2-position Euro-style terminal block
J6	1		10-position IDC header
J7,8	2		16-position IDC header
P0–2	3		2-position jumper shunt
R0–4	5	603-MFR-25FRF5210K	10K resistor
R5	1	604-MFR-24FRF521K	1K resistor
R6-8	3		Resistor array, isolated, 4-resistors*
R9,10	2		Resistor*
R _{TERM}	1		120 Ω resistor
U0	1		THVD1439 half-duplex RS-485 driver/receiver
U1	1		AT24C256 EEPROM
U2,3	2	595-ULN2003AIDR	ULN2003A NPN Darlington array
	1		9V 2A DC power supply

*R6–10 each limit current to the discrete LEDs. Choose their values appropriately for the supply voltage and current needs of the LEDs to be driven.

Table 1.4: Bill of Materials for AdaSheild Board

CHAPTER 2

PROTOCOL DESCRIPTION

I don't stand on protocol. Just call me
your Excellency.

—Henry Kissinger

THE CONTROL PROTOCOL used to display information on the readerboard sign is very simple. Commands are expressed largely in plain ASCII characters and are executed immediately as they are received.¹

In addition to plain, printable 7-bit ASCII characters, a few control codes are recognized as described below. String data may include any 8-bit value except as otherwise indicated.

Web Service API

The organization of the rest of this chapter is oriented toward a description of the protocol used to control the hardware directly over a USB or RS-485 connection. However, you can set up a central server which is connected to all the readerboard signs and busylight indicators in a small area (via USB and/or RS-485). This allows clients to send commands to the server over the LAN for control of the readerboards.

The protocol between clients and servers uses messages which are simple HTTP requests posted to the server's listening port URL. If the server were configured to run on `example.org` on port 43210, then all client requests to that server would begin with

`http://example.org:43210/readerboard/v1/<message>?a=<targets>`

¹Technically, they may even be executed *while* they are being received.